互斥同步与锁竞争优化实验报告

姓名：赵心瑞　　学号：24307140096

1 实现方案

原始物理内存分配器使用一条空闲页链表和一把全局锁，多 CPU 并发分配、释放时容易竞争。本实验在 `lock` 分支修改 `kernel/kalloc.c`，将其改为每 CPU 一条链表、一把自旋锁，并支持跨 CPU 偷取页面。

1. **数据结构与初始化：**将 `kmem` 改为大小为 `NCPU` 的数组，每个元素保存独立的锁和 `freelist`。`kinit()` 先初始化所有锁，再调用一次 `freerange()`。初始空闲页集中在执行初始化的 CPU 上。
2. **释放页面：**`kfree()` 保留地址检查和页面填充，通过 `cpuid()` 获取当前 CPU 编号，在对应锁保护下将页面插入本地链表头。
3. **分配与偷页：**`kalloc()` 优先从本地链表摘取页面。本地为空时，先释放本地锁，再依次检查其他 CPU 的链表，在目标锁保护下取走一页；遍历后仍未获得页面则返回 0。

通过配对的 `push_off()/pop_off()` 关闭本 CPU 中断并恢复原状态，使获取和使用 CPU 编号期间不会因时钟抢占而迁移。所有链表读写均持有对应锁；偷页时一次只持有一把锁，避免多锁循环等待。分配后的 `memset()` 在锁外执行，缩短临界区。

2 测试结果与分析

在 QEMU 三 CPU 环境下运行测试。`##test-and-set` 表示获取锁失败后的重试次数；下表统计各 `kmem` 锁的累计竞争次数或增量，结果截图见图1。

指标	原始实现	优化后
test1 竞争次数	32926	0
test3 新增竞争次数	43531	7502
test2 空闲页数量	32465	32465
kalloctest 结果	test1、test3 FAIL	test1–test3 全部 OK

原版 test3 增量为 $76457 - 32926 = 43531$，优化后为 $7502 - 0 = 7502$，减少约 82.8%。这次测试中，本地链表使常规操作避开全局锁竞争；跨 CPU 偷页仍需访问其他锁，因此竞争未完全消失。计数受运行时序影响，这里比较的是本次测试结果，而非运行时间加速比。

优化前后空闲页数量一致，test2 通过。`usertests sbrkmuch` 输出 `test sbrkmuch: OK`，`usertests -q` 最终输出 `ALL TESTS PASSED`（图2），表明本次实现通过大量内存申请及用户回归测试。回归截图中的部分用户态异常属于非法访问测试，其对应测试最终为 OK。

3 测试截图

优化前

```
init: starting sh
$ kalloctest
start test1
test1 results:
--- lock kmem/bcache stats
lock: kmem: #test-and-set 32926 #acquire() 433068
lock: bcache: #test-and-set 0 #acquire() 1274
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 83933 #acquire()
147
lock: kmem: #test-and-set 32926 #acquire() 433068
lock: proc: #test-and-set 5878 #acquire() 1411
lock: uart: #test-and-set 1298 #acquire() 117
lock: proc: #test-and-set 1163 #acquire() 402003
tot= 32926
test1 FAIL
start test2
total free number of pages: 32465 (out of 32768)
.....
test2 OK
start test3
..........child done 100000
--- lock kmem/bcache stats
lock: kmem: #test-and-set 76457 #acquire() 4232492
lock: bcache: #test-and-set 0 #acquire() 1374
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 83933 #acquire()
147
lock: kmem: #test-and-set 76457 #acquire() 4232492
lock: uart: #test-and-set 13668 #acquire() 1185
lock: proc: #test-and-set 6423 #acquire() 4217
lock: log: #test-and-set 2895 #acquire() 68
tot= 76457
2926
test3 FAIL m 32926 n 76457
$
```

优化后

```
hart 2 starting
hart 1 starting
init: starting sh
$ kalloctest
start test1
test1 results:
--- lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 203188
lock: kmem: #test-and-set 0 #acquire() 117450
lock: kmem: #test-and-set 0 #acquire() 112484
lock: bcache: #test-and-set 0 #acquire() 1274
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 71689 #acquire()
147
lock: pr: #test-and-set 9361 #acquire() 5
lock: uart: #test-and-set 3439 #acquire() 117
lock: proc: #test-and-set 1173 #acquire() 880
lock: proc: #test-and-set 284 #acquire() 401443
tot= 0
test1 OK
start test2
total free number of pages: 32465 (out of 32768)
.....
test2 OK
start test3
..........child done 100000
--- lock kmem/bcache stats
lock: kmem: #test-and-set 7502 #acquire() 3756795
lock: kmem: #test-and-set 0 #acquire() 346466
lock: kmem: #test-and-set 0 #acquire() 254296
lock: kmem: #test-and-set 0 #acquire() 58
lock: kmem: #test-and-set 0 #acquire() 58
lock: kmem: #test-and-set 0 #acquire() 58
lock: kmem: #test-and-set 0 #acquire() 58
lock: kmem: #test-and-set 0 #acquire() 58
lock: bcache: #test-and-set 0 #acquire() 1374
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 71689 #acquire()
147
lock: uart: #test-and-set 52414 #acquire() 1329
lock: pr: #test-and-set 9361 #acquire() 5
lock: kmem: #test-and-set 7502 #acquire() 3756795
lock: log: #test-and-set 2826 #acquire() 68
tot= 7502

test3 OK
$
```

图 1: kalloctest 优化前后对比。

```
$ usertests sbrkmuch
usertests starting
test sbrkmuch: OK
ALL TESTS PASSED
```

```
test nowrite: usertrap(): unexpected scause 0xf pi
d=6572
            sepc=0x1e5e stval=0x0
usertrap(): unexpected scause 0xf pid=6573
            sepc=0x1e5e stval=0x80000000
usertrap(): unexpected scause 0xf pid=6574
            sepc=0x1e5e stval=0x3fffffe000
usertrap(): unexpected scause 0xf pid=6575
            sepc=0x1e5e stval=0x3ffffff000
usertrap(): unexpected scause 0xf pid=6576
            sepc=0x1e5e stval=0x4000000000
usertrap(): unexpected scause 0xf pid=6577
            sepc=0x1e5e stval=0xffffffffffffffff
OK
test pgbug: OK
test sbrkbugs: usertrap(): unexpected scause 0xc p
id=6580
            sepc=0x4d86 stval=0x4d86
usertrap(): unexpected scause 0xc pid=6581
            sepc=0x4d86 stval=0x4d86
OK
test sbrklast: OK
test sbrk8000: OK
test badarg: OK
ALL TESTS PASSED
$
```

图 2: usertests sbrkmuch 通过，以及 usertests -q 最终全部通过。